

Architetture Software: Mega Riassunto per l'Esame Teorico

Fondamenti (Evoluzione e Obiettivi)

- **Monolite:** Un'unica grande applicazione su un solo server. L'approccio tradizionale, difficile da aggiornare.
 - **Microservizi:** Architettura moderna. L'applicazione è divisa in **servizi piccoli, indipendenti** e autonomi.
 - **Cloud-native:** Un sistema progettato fin dall'inizio per funzionare al meglio in ambienti cloud (es. usando container).
 - **Scalabilità Orizzontale (Scaling Out):** La strategia migliore. Significa **aggiungere più server** identici per gestire il carico crescente.
 - **Scalabilità Verticale (Scaling Up):** Aumentare le risorse (CPU, RAM) di un **singolo server**. Limite fisico e costi alti.
 - **Resilienza:** La capacità di **riprendersi** da un guasto e tornare a funzionare.
 - **Fault Tolerance:** La capacità di **continuare a funzionare** anche se ci sono guasti.
 - **High Availability (HA):** L'obiettivo di essere **sempre disponibili** agli utenti.
-

Dati e Teoria (CAP e Modelli Transazionali)

- **Teorema CAP:** Un sistema distribuito può garantire al massimo due tra: **Consistenza (C), Disponibilità (A), Tolleranza alle Partizioni (P)**. La P è quasi sempre obbligatoria.
 - **ACID:** Modello dei database relazionali (SQL). Garantisce la **Consistenza Forte** (i dati sono sempre perfetti). **Atomicity-Consistency-Isolation-Durability**
 - **BASE:** Modello dei database NoSQL. Privilegia la **Disponibilità**, accettando che la **Consistenza** sia raggiunta **Eventualmente** (non subito). **Basically Available-Soft State-Eventual Consistency**
-

Comunicazione e Disaccoppiamento

- **HTTP Stateless:** Il protocollo web standard, in cui **ogni richiesta è indipendente** dalle precedenti. È essenziale per scalare orizzontalmente.
- **Disaccoppiamento Architeturale:** Mantenere i servizi **il più indipendenti possibile** l'uno dall'altro.
- **Comunicazione Asincrona:** I servizi comunicano **senza aspettarsi una risposta immediata**. Aumenta la resilienza.
- **Message Queues / Brokers:** Sistemi (come RabbitMQ o Kafka) che funzionano da buffer, gestendo la comunicazione asincrona.
- **Event-driven (Publish/Subscribe):** Un servizio invia un **evento** ("qualcosa è successo") e gli altri **reagiscono** ad esso.

Strumenti di Resilienza e Gestione

- **Load Balancers:** Distribuiscono il traffico in entrata sui server, ottimizzando il carico e garantendo **HA**.
- **Reverse Proxy:** Si trova davanti ai server e offre sicurezza, caching e centralizzazione.
- **Stateless Authentication:** L'autenticazione che **non memorizza lo stato** della sessione sul server.
- **JWT (JSON Web Token):** Token che contiene i dati dell'utente, firmato e **autocontenuto**, perfetto per l'autenticazione stateless.
- **Circuit Breakers (Interruttori):** Isolano un servizio che sta fallendo, bloccando le chiamate per un periodo, per dargli tempo di recuperare e prevenire il collasso a catena.
- **Retry Patterns:** Tentativi di **ripetere** una chiamata in caso di errori temporanei (es. problemi di rete).

Qualità e Operazioni (DevOps/SRE)

- **Observability:** La capacità di **capire cosa succede** all'interno del sistema analizzando i dati esterni (i 3 pilastri).
 - **Metrics:** Primo pilastro. **Numeri aggregati** nel tempo (es. latenza media).
 - **Logs:** Secondo pilastro. **Eventi discreti** e testuali (es. un errore specifico).
 - **Traces:** Terzo pilastro. Permettono di **seguire il percorso** di una singola richiesta attraverso tutti i microservizi.
 - **Chaos Engineering: Testing in Produzione.** Iniettare attivamente guasti (**Fault Injection**) per testare la vera resilienza.
 - **Blameless Culture:** Approccio a un incidente dove ci si concentra sull'**imparare dal fallimento**, non sull'identificare i colpevoli.
 - **RTO (Recovery Time Objective):** Il **tempo massimo** in cui il sistema può stare giù dopo un disastro.
 - **RPO (Recovery Point Objective):** La **quantità massima di dati** che si è disposti a perdere in un disastro.
 - **Cognitive Load:** La quantità di **complessità** che il cervello di un team deve gestire. L'architettura deve minimizzarla.
-

Requisiti Enterprise e Cloud

- **Compliance (GDPR/SOX):** Conformità a **leggi e regolamenti** (es. privacy o report finanziari).
- **Audit Trails:** Registri che tracciano **chi ha fatto cosa**, utili per la sicurezza e la conformità.
- **Cost Optimization:** Ridurre i costi del Cloud, ad esempio usando **Right-sizing** (allocare solo le risorse necessarie) o **Auto-scaling** (ridimensionamento automatico).

L'**architettura three-tier** (o **architettura a tre livelli/strati**) è un modello consolidato di architettura software e hardware che suddivide un'applicazione in **tre livelli logici e spesso fisici distinti**.

Questo modello è stato a lungo l'architettura predominante per le applicazioni client-server e offre vantaggi in termini di **modularità, scalabilità, manutenibilità e sicurezza**.

I tre livelli sono:

1. **Livello di Presentazione (Presentation Tier / Tier 1):**
 - **Funzione:** È l'interfaccia utente (UI) con cui l'utente interagisce. Gestisce la visualizzazione dei dati e la raccolta dell'input.
 - **Esempio:** In un'applicazione web, è rappresentato dal **server web** e dai contenuti visualizzati nel browser (HTML, CSS, JavaScript).
2. **Livello di Business Logic (Application Tier / Business Logic Tier / Tier 2):**
 - **Funzione:** Contiene la logica di business dell'applicazione. Elabora le richieste dell'utente, applica le regole aziendali, coordina il flusso di lavoro e funge da intermediario tra gli altri due livelli. È l'unico che può comunicare con il Livello Dati.
 - **Esempio:** In un'applicazione web, è rappresentato dall'**application server** che esegue codice (es. Python, Java, PHP) per processare i dati (ad esempio, calcolare l'IVA su un ordine o interrogare la disponibilità di un prodotto).
3. **Livello Dati (Data Tier / Tier 3):**
 - **Funzione:** Responsabile della memorizzazione, gestione e recupero dei dati persistenti.
 - **Esempio:** È rappresentato dal **server di database** (es. MySQL, PostgreSQL, Oracle).

Vantaggi principali:

- **Modularità:** Ogni livello può essere sviluppato, gestito e aggiornato in modo indipendente dagli altri.
- **Scalabilità:** È possibile scalare i livelli separatamente, ad esempio aggiungendo più application server senza toccare il database o il front-end.
- **Sicurezza:** Il livello di presentazione non comunica direttamente con il database; la comunicazione passa attraverso il Livello Logica Applicativa, che può fungere da "firewall interno" e impedire attacchi diretti come le SQL Injection.

Caratteristiche Generali del Protocollo HTTP

L'**HyperText Transfer Protocol (HTTP)** è il protocollo fondamentale utilizzato per lo scambio di informazioni sul World Wide Web. È un protocollo di livello applicativo che funziona secondo il modello **client-server**.

Le sue caratteristiche principali sono:

- **Protocollo Client-Server:** La comunicazione inizia sempre con un **client** (es. un browser web) che invia una **richiesta** (Request) a un **server**, il quale elabora la richiesta e invia una **risposta** (Response).
 - **Basato su TCP/IP:** HTTP si appoggia tipicamente sul protocollo TCP (Transmission Control Protocol) per garantire che i pacchetti di dati arrivino al destinatario in modo affidabile e ordinato.
 - **Utilizzo di Porte:** La porta standard di default per HTTP è la **80**, mentre per la sua versione sicura, **HTTPS**, è la **443**.
 - **Metodi (Verbi):** Definisce un insieme di metodi per indicare l'azione desiderata sulla risorsa identificata. I più comuni sono **GET** (richiesta dati), **POST** (invio dati), **PUT** (sostituzione) e **DELETE** (eliminazione).
 - **URL (Uniform Resource Locator):** Utilizza gli URL per identificare in modo univoco le risorse sul web (pagine, immagini, API, ecc.).
-

I Codici di Stato HTTP (200, 300, 400)

Ogni risposta HTTP include un codice di stato numerico che indica l'esito della richiesta:

1. **200 (OK - Successo):**

- Indica che la richiesta del client è stata **ricevuta, compresa e accettata con successo**.
- È la risposta standard per una richiesta **GET** riuscita, e il corpo della risposta contiene la risorsa richiesta (es. la pagina web).

2. **300 (Multiple Choices - Reindirizzamento):**

- Indica che la risorsa richiesta si trova in più posizioni. Sebbene il 300 non sia comunemente usato, è l'inizio della classe 3xx (Redirect).
- Il server comunica al client di **reindirizzare la sua richiesta** verso un nuovo URL (es. 301 per spostamento permanente, 302 per spostamento temporaneo).

3. **400 (Bad Request - Errore del Client):**

- Indica che il server **non può elaborare la richiesta** perché c'è un **errore percepito nella sintassi o nella validità della richiesta stessa** inviata dal client.
 - Significa che il client deve **modificare la richiesta** prima di riprovare (a differenza del 404 che indica solo che la risorsa non è stata trovata).
-

Stateful e Stateless (Contesto Architetture)

Questi concetti definiscono come un sistema gestisce le informazioni del client tra richieste consecutive.

1. Stateless (Senza Stato)

- **Definizione:** Il sistema (tipicamente il server web) **non conserva alcuna informazione** sul contesto o sulla sessione delle interazioni precedenti del client.
- **Interazioni:** Ogni richiesta è **completamente indipendente** e deve contenere tutte le informazioni necessarie per essere elaborata.
- **Vantaggio HTTP:** HTTP è per sua natura **stateless**. Questo permette una maggiore **scalabilità orizzontale** (le richieste possono essere distribuite a qualsiasi server) e una migliore **affidabilità**, poiché il guasto di un singolo server non comporta la perdita di dati di sessione.

2. Stateful (Con Stato)

- **Definizione:** Il sistema **mantiene e memorizza** le informazioni (lo "stato") sulle interazioni passate del client.
- **Interazioni:** Le richieste successive **dipendono** da questo stato conservato per poter procedere (es. l'autenticazione, il contenuto di un carrello)
- **Gestione dello Stato:** Quando si desidera un'esperienza continua sul web (che è basata su HTTP stateless), lo stato viene gestito attraverso meccanismi esterni al server stesso, come l'uso di **cookie** o **token** inviati e ricevuti con ogni richiesta, oppure salvando i dati di sessione in un **database centralizzato**.